

ADVANCED SQL

Case Studies Manual

Applying Joins, Subqueries, Views, Stored Procedures, Triggers & Window Functions to Real Business Problems

Session Format	Instructor-Led, Live SQL
Duration	2 Hours
Level	Intermediate → Advanced
Database Engine	MySQL 8.0 (MySQL Workbench)
Audience	Developers, Data Analysts & Students
Prerequisite	Completion of Core SQL Fundamentals (Topics 1–17)

Table of Contents

1. Introduction & Learning Objectives.....	3
2. Session Agenda (2-Hour Breakdown).....	4
3. Practice Database: Schema Overview	5
4. Case Study 1 —The Complete Order Report.....	7
5. Case Study 2 —Finding Top Spenders	7
6. Case Study 3 —Category Performance.....	7
7. Case Study 4 —A Reusable Sales Dashboard	8
8. Case Study 5 —Automating Order Placement	8
9. Case Study 6 —Building an Inventory Audit Trail.....	8
10. Case Study 7 —Ranking & Running Totals.....	8
11. Case Study 8 —Fixing a Flat Denormalized Table.....	9
12. Self-Practice Exercises	9

1. Introduction & Learning Objectives

Welcome back! Over your earlier sessions you built a strong foundation in SQL — from writing your first CREATE TABLE statement to filtering, sorting, aggregating and joining data. This manual is different: it does not re-teach syntax. Instead, it takes everything you already know and applies it to realistic, end-to-end business problems, the way you would encounter them on the job as a developer or data analyst.

Every case study in this manual is built around a single, consistent e-commerce database, so that the relationships between tables, and the way techniques build on each other, feel like a real project rather than a set of disconnected exercises.

Who This Session Is For

- Software developers who write application queries and need to reason about performance and correctness.
- Data analysts who need to turn raw transactional tables into decision-ready reports.

Learning Objectives

By the end of this session, you will be able to:

- Analyze real-world business requirements.
- Interpret database schemas and table relationships.
- Design SQL queries to solve business problems.
- Select the most appropriate SQL approach based on the business requirement.
- Write clear, accurate, and efficient SQL queries.
- Validate query results and explain your approach.
- Develop confidence in solving interview-style SQL challenges.

Prerequisite Check

This manual assumes you are comfortable with: SELECT/WHERE/ORDER BY, INSERT/UPDATE/DELETE, basic JOIN syntax, GROUP BY with aggregate functions, and basic string/date functions. If any of these feel shaky, flag it with your instructor before we begin — the case studies move quickly.

2. Session Agenda (2-Hour Breakdown)

This session is intentionally paced so that every technique is demonstrated live, then immediately practiced by you before we move on.

Time	Segment
0:00 – 0:10	Kickoff & Schema Walkthrough
0:10 – 0:25	Case Study 1
0:25 – 0:40	Case Study 2
0:40 – 0:50	Case Study 3
0:50 – 1:00	Case Study 4
1:00 – 1:10	Break
1:10 – 1:25	Case Study 5
1:25 – 1:40	Case Study 6
1:40 – 1:55	Case Study 7
1:55 – 2:00	Case Study 8 (Overview) & Wrap-up

Facilitation Tip

Case Study 8 (Normalization) is included as a reference walkthrough for self-study — with a 2-hour clock, the instructor will summarize it live and let you work through the full detail afterward.

3. Practice Database: Schema Overview

All case studies in this manual run against a single database named shopnest, modeled on a small e-commerce business. Six tables cover customers, products, orders, order line items, employees and departments — enough breadth to demonstrate every advanced concept without overwhelming complexity.

Entity-Relationship Summary

- One Customer can place many Orders (1 : N)
- One Order can contain many Order_Items; one Product can appear on many Order_Items (Orders : Products is M : N, resolved by Order_Items)
- One Department can have many Employees (1 : N); an Employee may manage other Employees (self-referencing FK)

Table: customers

Column	Data Type	Constraint
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE
city	VARCHAR(50)	
state	VARCHAR(50)	
signup_date	DATE	DEFAULT (CURRENT_DATE)

Table: products

Column	Data Type	Constraint
product_id	INT	PRIMARY KEY, AUTO_INCREMENT
product_name	VARCHAR(100)	NOT NULL
category	VARCHAR(50)	NOT NULL
price	DECIMAL(10,2)	CHECK (price > 0)
stock_qty	INT	DEFAULT 0

Table: orders

Column	Data Type	Constraint
order_id	INT	PRIMARY KEY, AUTO_INCREMENT
customer_id	INT	FOREIGN KEY → customers(customer_id)
order_date	DATE	DEFAULT (CURRENT_DATE)

Column	Data Type	Constraint
status	VARCHAR(20)	DEFAULT 'PENDING'

Table: order_items

Column	Data Type	Constraint
order_item_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FOREIGN KEY → orders(order_id)
product_id	INT	FOREIGN KEY → products(product_id)
quantity	INT	CHECK (quantity > 0)
unit_price	DECIMAL(10,2)	NOT NULL

Table: employees & departments

Column	Data Type	Constraint
employee_id	INT	PRIMARY KEY, AUTO_INCREMENT
first_name / last_name	VARCHAR(50)	NOT NULL
department_id	INT	FOREIGN KEY → departments(department_id)
manager_id	INT	FOREIGN KEY → employees(employee_id) — self join
salary	DECIMAL(10,2)	CHECK (salary > 0)
hire_date	DATE	NOT NULL

Schema creation script (run once at the start of the session)

4. Case Study 1 —

CASE STUDY 1: The Complete Order Report

🕒 Suggested time: 15 minutes |

Business Scenario

The operations manager at ShopNest wants a single report that shows every order with the customer's name, the products ordered, quantities and line totals — so nothing forces her to open three different spreadsheets to understand one order.

Business Requirements

1. Produce a report with customer name, order ID, order date, product name, quantity and line total (quantity × unit_price) for every order.
2. Extend the report so that customers who signed up but have never placed an order still appear, with blank order details.

5. Case Study 2 —

CASE STUDY 2: Finding Top Spenders

🕒 Suggested time: 15 minutes |

Business Scenario

Marketing wants to identify customers whose total spend is above average, and separately, the single highest-spending customer in each state, so they can design a tiered loyalty program.

6. Case Study 3 —

CASE STUDY 3: Category Performance Review

🕒 Suggested time: 10 minutes |

Business Scenario

The category manager wants to discontinue underperforming product categories. She defines "underperforming" as any category with total revenue below ₹50,000 or fewer than 10 total units sold.

7. Case Study 4 —

CASE STUDY 4: A Reusable Sales Dashboard

🕒 Suggested time: 10 minutes |

Business Scenario

The Case Study 3 query is exactly what the finance team wants to see every Monday morning, but they use Excel, not SQL. Rather than emailing them a query to paste in every week, we package it as a View — a saved, reusable virtual table.

8. Case Study 5 —

CASE STUDY 5: Automating Order Placement

🕒 Suggested time: 15 minutes |

Business Scenario

Right now, placing an order requires three separate manual steps: insert into orders, insert into order_items, then reduce stock_qty in products. Any developer who forgets one step corrupts the data. We will wrap the entire operation in a single Stored Procedure so the application only ever makes one call.

9. Case Study 6 —

CASE STUDY 6: Building an Inventory Audit Trail

🕒 Suggested time: 15 minutes |

Business Scenario

The warehouse team was recently blindsided by a stock discrepancy and cannot tell who changed a product's stock_qty or when. We will build an automatic audit log using triggers, so every change is captured with zero extra application code.

10. Case Study 7 —

CASE STUDY 7: Ranking Customers & Running Revenue Totals

🕒 Suggested time: 15 minutes |

Business Scenario

Leadership wants three things for the quarterly review: (1) each customer's spend rank within their own state, (2) a running total of company-wide revenue by date, and (3) a 3-order moving average of order value to smooth out spikes. All three are natural fits for window functions, because — unlike GROUP BY — they let us keep every original row while still computing group-aware calculations.

11. Case Study 8 —

CASE STUDY 8: Fixing a Flat, Denormalized Table

 Suggested time: 10 minutes |

Business Scenario

A well-meaning teammate built the very first version of the orders system as one giant spreadsheet-like table. It works, but every insert duplicates customer and product details, and every update risks leaving the data inconsistent. We will walk through normalizing it step by step — the same journey that produced the customers / products / orders / order_items schema you have been using all session.

The Problem Table

order_id	customer_name	customer_city	product_name	product_category	quantity	unit_price
2001	Aarav Sharma	Pune	Wireless Mouse, USB-C Cable	Electronics, Accessories	2, 1	450, 250
2002	Priya Nair	Chennai	Mechanical Keyboard	Electronics	1	3200

12. Self-Practice Exercises

Attempt these independently after the session, using the shopnest schema from Section 3. Solutions are intentionally not provided — discuss your approach with a peer or your instructor.

- List every product that has never appeared in an order_items row.
- Show each employee alongside their department name and their manager's full name.
- Find customers who have spent more than the average spend of customers in their own city (correlated subquery).
- Find products priced higher than every product in the "Accessories" category (ALL).
- List customers who have placed more than 3 orders, along with their total spend.
- Find product categories where the average unit price exceeds ₹2,000.
- Create vw_low_stock_products showing all products with stock_qty below 10.

10. Write `sp_cancel_order(IN p_order_id INT)` that sets an order's status to CANCELLED and restores `stock_qty` for every item on that order.
11. Write a BEFORE INSERT trigger on orders that rejects (via SIGNAL) any order where `customer_id` does not exist — even though the foreign key already prevents this, practice raising a custom error message.
12. For every customer, compute the number of days between their `signup_date` and their first `order_date` using a window function or subquery, and rank customers by how quickly they made their first purchase.
13. A colleague proposes storing "top_3_products" as a single comma-separated column on the customers table. Explain, using normal form terminology, why this is a poor design choice.

© Shalinee Priya